
CS 121

Computer Science I

Instructor:
Dr. Wang, Jingwen

Lecture 3:
Values and Data Types

Lecture outline

Announcements/notes

- Office Hours and CS Tutoring Nights
- TAs and Lab sessions
- Start Reading for Week 2

Today's lecture

- Review from last time
- Values, Data Types

Review: Von Neumann Architecture

- Basic layout of all modern computers

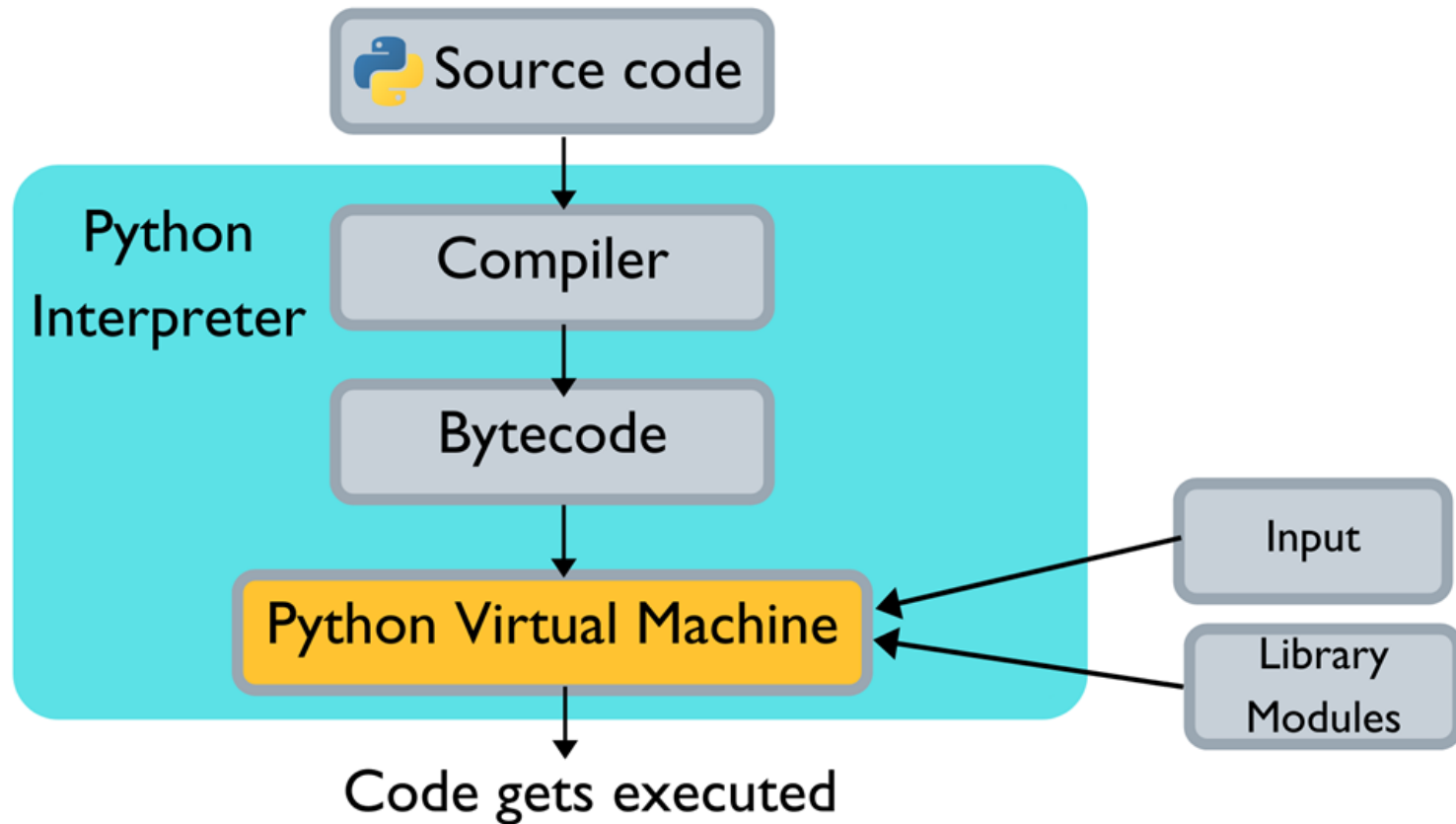


Review: Python Interpreter

- What's an interpreter?
- Turns the (human readable) programming language into intermediate language, which is again translated into the (computer readable) instructions that a computer executes
- For Python, we'll be using Anaconda Python Distribution (which contains a Python interpreter and many other Python libraries)

Reviews: How Does Python Work?

■ The interpreter workflow



Review: Our First Program

- Type the following to any of the Python environment mentioned above

```
print("Hello world!")
```

- The result is a printed message on your output console.

```
Hello world!
```

Exercise

- Print the following message:
 - Why do gorillas have big nostrils? Big finders!!
- What's wrong with the following script:
 - `print("What is pink and fluffy?")`
 - `print('He said, "Aren't can't shouldn't wouldn't"')`

Other uses of the `print()` function

What do the following do in Python?

- `print('Hello world!')`
- `print("Hello" + "world!")`
- `print("Hello", "world!")`
- `print('Alice said, "hi"')`
- `print("Alice said, 'hi'")`
- `print(7+2)`
- `print('answer = ' + 7 + 2)`
- `print('answer = ', 7 + 2)`

Comments

```
#-----  
# lecture2.py  
# Author: Dr. Wang  
# Date: Jan. 20, 2022  
# Purpose: Demo the use of IDEs  
#-----  
  
print("Hello world!")    # Isn't this easy
```

Writing Comments

Comment Syntax	Comment Type
<code># comment here</code>	In-line comment
<code>""" comment here and here """</code>	Multi-line comment
<code>''' comment here and here '''</code>	Multi-line comment

Header Comments Example 1

```
#-----  
# lecture2.py  
# Author: Dr. Wang  
# Date: Jan. 20, 2022  
# Purpose: Demo the use of IDEs  
#-----  
  
print("Hello world!")    # Isn't this easy
```

Header Comments Example 2

```
"""
```

```
lecture2.py
```

```
Author: Dr. Wang
```

```
Date: Jan. 28, 3030
```

```
Purpose: Demo the use of IDEs
```

```
"""
```

```
print("Hello world!")    # Isn't this easy
```

VALUES & DATA TYPES

Values

- A value is like a number or a word that a program manipulates.
 - 5
 - 3.14
 - "Hello world!"
- We can specify values directly in the programs we write.
- For example, we can specify a number as a **literal** just by (literally) typing it directly into the program (e.g., 5 or 4.32)

Expressions

- An **expression** is a piece of code that has **a value**.
- **Examples:**
 - Literals: `2`, `4.59`, `"Python"`
 - Arithmetic expressions: `3 * (5 + x)`
 - `(5 + x)` is itself an expression
 - And so are `x` and `5`
 - It's expressions built of expressions!

Basic Arithmetic

- Some arithmetic operators in Python are:
 - `**` (exponentiation, “raise to the power of”)
 - `*` (multiply), `/` (divide)
 - `+` `-` (addition, subtraction)
- These are listed in order from higher **precedence** to lower **precedence**
- Of course you can use parentheses to make the order you want explicit:

`0.5 * (3.14 + 100) / 100`

Data Types

- ❑ Inside the computer, everything is expressed **in bits**.
- ❑ A **data type** decides **how to interpret these bits**, and **what we can do with them**.
- ❑ **Every expression** in Python has a **data type**.

Data Types

- Some of the built-in types are:

Type	Description	Examples
int	Integer numbers	2, -44, 0
float	Floating-point numbers	3.0, -0.1, 6.22e23
bool	Boolean (True/False) values	True, False
str	Strings of characters	"hi", "1234", "2@5"
list	Lists of values	["Prisoner", 7] [2, 3, 4, 5, 7]

Integers

The data type **int** represents integers: whole numbers that are **positive, zero or negative**.

- Literal integers: a sequence of digits, like 2341
 - With no leading zeros!
 - 0 by itself is okay, **007** is not.
- In Python, integers have **no stated limit to their size**.
 - They can have as many digits as you have memory for.
 - That is not true for most languages, like C++ and Java.

Floating-point

The data type called **float** represents floating-point numbers, numbers with a decimal point.

- In a computer, they have limited precision and range.
- Two forms of literal floating-point numbers:
 - A number with a decimal point: 3.14, .027, 1., 0.1
 - Must have a decimal point!
 - 1.0 or 1. is a float, 1 is an integer
 - Scientific notation (“E” notation)
 - 6.022e23, 1.0E9, 31e-2
 - The “e” represents “times 10 to the ...”
 - Does not have to have a decimal point, the e is enough to make the value a float
 - The exponent must be an integer

Floating-point limitations

- Lots of floats are stored with approximation.
- The exact limits are on the number of bits, not digits.
 - Even 0.1 can't be represented exactly **in binary**
 - Try: $0.1 + 0.1 + 0.1$
 - It gives 0.30000000000000004
- Note that this is NOT the fault of a flaw in the hardware or software or language or OS. It is inherent in trying to store numbers in a finite machine.
- What to take away from all this? Don't expect exact numbers using floating point representation.
 - ```
if ((0.1+0.1+0.1) != 0.3) :
 print("They are NOT equal")
```

# Arithmetic on integers and floats

You can perform arithmetic on both ints and floats. For most arithmetic operators (+ - \* \*\*) the rules are:

- If both operands are ints, the result is an int.
  - $3 + 5 \rightarrow 8$
  - $2 ** 100 \rightarrow 1267650600228229401496703205376$
- If one operand is a float or both are floats, the result is a float. → Automatic type conversion.
  - $3.0 + 0.14 \rightarrow 3.14$
  - $100 - 1.0 \rightarrow 99.0$
  - $2.0 ** 100 \rightarrow 1267650600228229401496703205376.0$
- There is ONE exception...
  - What should  $1 / 2$  result in?

# Division

Python actually has *two* division operators, / and //.

- / always gives a **float** no matter what type of operands it has.
  - $1 / 2 \rightarrow 0.5$
  - $6 / 3 \rightarrow 3.0$
  - $3.0 / 0.5 \rightarrow 6.0$
- // does **floor division**: rounds the answer down to a whole number.
  - If both operands are integers, so is the result.
    - $22 // 7 \rightarrow 3$
    - $1 // 2 \rightarrow 0$
  - If either operand is a float, so is the result.
    - But it still has a **whole-number** value.
    - $22 // 7.0 \rightarrow 3.0$
    - $3.1 // 0.5 \rightarrow 6.0$
- With either operator, dividing by **zero** cause an error!

# Remainder (modulo)

The **% operator** (modulo or mod) finds the remainder of a division.

- Its possible results are between **0 (inclusive) and the right hand side operand (exclusive)**. Example: for  $x \% 3$ , the only results are 0, 1, or 2.
  - $6 \% 3 \rightarrow 0$
  - $7 \% 3 \rightarrow 1$
  - $8 \% 3 \rightarrow 2$
  - $9 \% 3 \rightarrow 0$
- **Uses for modulo operator:**
  - Even/odd:  $n$  is even if  $n \% 2$  is zero
  - Picking off digits:  $n \% 10$  is the last (rightmost) digit of  $n$
  - “Clock arithmetic”
    - Minutes are mod 60:  $3:58 + 15 \text{ minutes} = 4:13$
    - Hours are mod 12:  $10:00 + 4 \text{ hours} = 2:00$
- **Python can do modulo on floats.**
  - $5 \% 2.4 \rightarrow 0.2$  (remainder after 2.4 goes into 5 two times, with remainder 0.2)
  - But it is far, far more common with integers.



# A common error

- In algebra it is perfectly normal to write things like “2x” or “4ac”. The operator is implied.
  - It’s multiplication!
- In Python this does not work at all. Both of those expressions would be rejected as **invalid identifiers, not as multiplied variables.**
- You **MUST** put an asterisk \* where you mean two things to be multiplied! Even an expression like “2(a + c)” will not work without an operator!  
You must write it as:  $2 * (a + c)$

# The ^ (caret) operator

- A lot of math books will use  $^$  to mean “raised to the power of” or sometimes, “times 10 to the power of”
- This is NOT the same as the  $**$  operator in Python.
- The  $^$  operator in Python is a binary XOR operator, working on individual bits of a number. Definitely does NOT do the same thing as  $**$
- Thus, if you use an expression like  $10^3$ , you get 9, not 1000!
- But because it is a valid operator in Python, you get no kind of warning or error message.
- Be aware!

# Rounding

One more numeric function, builtin – so you do NOT have to import math library to use it

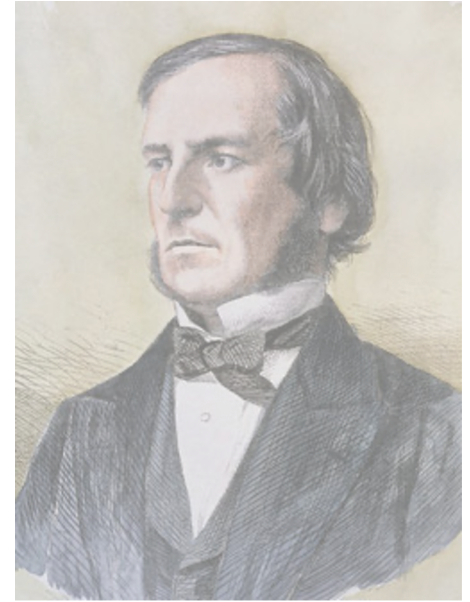
- **round** has **either** one or two arguments
  - If it has just **ONE** argument, it will **round the argument to the nearest integer**
    - `round(5.2) → 5`
    - `round(7.9) → 8`
  - If it has **TWO** arguments, the **second** one is the **number of decimal places desired**. The first argument's value will be rounded to that number of decimals
    - `round(math.pi, 2) → 3.14`
    - `round(2.71818, 0) → 3.0`
    - `round(12, -1) → 10`

# Precedence of operators

- ❑ There are many, many operators in Python!
- ❑ The few which we have seen are listed in priority order, from highest to lowest.
  - ❑ \*\*
  - ❑ \*, /, //, %
  - ❑ +, -
- ❑ You should learn what each operator does semantically, what types of operands it works ON and what type or types it **RETURNS**

# Booleans

- The data type **bool** represents boolean values.
  - It is named after George Boole, English mathematician and logician.
- The data type has exactly two values: **True** and **False**
  - No quotes! They are not strings.
  - Case sensitive: capital T and F
- You **don't do arithmetic with the values**
  - The operators you DO use with them are **and**, **or** and **not**.
  - Most often used with **if** and **while** statements.



# Strings

- The data type `str` represents **strings**: sequences of characters.
- Literal strings: a sequence of characters in **single** or **double** quotes.
  - `'hello', "world", ""` (empty string)
  - Use whichever quote isn't in the string:
    - `'some "quotes"', "O'Conner"`
- Can perform some operations on strings:
  - **Concatenate** (stick together) strings with a plus (+):
    - `"Hello, " + "Alice"`
  - Repeat a string by “replicating” with an **integer** and a **\***:

```
print('*' * 4) # prints ****
print(2 * "do") # prints dodo
```

# Escaped Characters

- If you have to include a single quote character in a string that is delimited by single quotes, **escape** it using a backslash:

```
print('the word "don\'t" is 5 chars long')
```

- The escape character "\" says to Python, **“treat the next symbol specially, not in the normal way”**.
- You have to escape backslashes, too:  

```
print("C:\\Python 3.4")
```
- There are some special characters which are useful in strings: tab "\t" and newline "\n"
  - All escaped characters are **ONE character** each, even though they are written with two (counting the backslash). Example:  
"\n\n\n" contains THREE characters.

# Exercise 1

- Write a program that reads in seconds, minutes, and hours as input, and outputs the time in seconds only.
- Ex: If the input is:
  - 40
  - 6
  - 1
- where 40 is the number of seconds, 6 is the number of minutes, and 1 is the number of hours, the output is:
  - 4000 seconds



## Exercise 2

- Starbucks offers buy 10 get 1 free for their Coffee
  - I have bought 23 cups of coffee this month.
  - How many free cups can I get?
- 
- How many cups left after redeem this offer?

# Exercise 3

- People find it easier to read time in seconds, minutes, and hours rather than just seconds.
- Write a program that reads in seconds as input, and outputs the time in seconds, minutes, and hours.
- Ex: If the input is:
  - 4000
- the output is:
  - Seconds: 40
  - Minutes: 6
  - Hours: 1

---

# Final notes



## Next time:

Function Calls and Built-in Functions



## Reminders:

Start Reading for Week 2